

MenuMate

Architecture Documentation

v1.0 · May 2026

Next.js 15	Expo SDK 52	Node / Express	PostgreSQL	Claude AI
------------	-------------	----------------	------------	-----------

AI-powered meal planning platform · Web · Mobile · Admin

Table of Contents

1.	Monorepo vs Polyrepo — decision	3
2.	Repository structure	4
3.	Technology stack	5
4.	Database schema (PostgreSQL)	6
5.	API endpoints	9
6.	Admin panel — scope	12
7.	AI integration	13
8.	Environment & deployment	14

1. Monorepo vs Polyrepo

For a fullstack pet project with shared types, utilities, and a single team, **monorepo is the recommended choice**. Here's why:

	Monorepo (recommended)	Polyrepo
Shared types	Single source of truth (TypeScript)	Duplication or separate npm package
Refactoring	One PR touches all layers	Cross-repo PRs, version bumps
Setup complexity	Medium (Turborepo)	Low per-repo, high overall
CI/CD	One pipeline, affected builds	Multiple pipelines
Code sharing	Native — just import	Publish to npm or use git submodules
Best for	1-3 devs, pet project, startup	Large teams, independent release cycles

Verdict: use Turborepo monorepo

- Shared TypeScript types between backend, web, and mobile
- Single lint/test/build config — less boilerplate
- Turborepo caches builds intelligently — fast CI even with 3 apps
- Easy to split into polyrepo later if the project grows

2. Repository Structure



Key packages installed at root level:

Tool	Purpose
Turborepo	Monorepo build orchestration, caching
pnpm workspaces	Package manager with hoisting
Prisma	ORM — schema, migrations, typed client
Zod	Runtime validation, shared between API and clients
ESLint + Prettier	Unified code style across all apps
Vitest	Unit tests (fast, ESM-native)
Playwright	E2E tests for web

3. Technology Stack

Web (Next.js 15)

- App Router + React Server Components
- Tailwind CSS v4 for styling
- NextAuth.js v5 for authentication
- SWR or TanStack Query for data fetching
- Recharts for nutrition analytics

Mobile (Expo SDK 52)

- Expo Router (file-based navigation)
- NativeWind (Tailwind for RN)
- Expo Camera — barcode scanning
- Expo Notifications — push via FCM/APNs
- MMKV for local storage (fast, offline)

Backend (Express + Node)

- TypeScript + Express 5
- Prisma ORM + PostgreSQL
- JWT + refresh tokens
- Bull + Redis — background jobs (AI, email)
- Multer + S3 — image uploads

4. Database Schema (PostgreSQL)

All tables managed via Prisma. Diagram shows primary keys (PK), foreign keys (FK), and key relationships.

Table: users

Column	Type	Notes
id	UUID PK	Primary key
email	VARCHAR(255) UNIQUE	Login identifier
password_hash	VARCHAR	bcrypt hash, nullable (OAuth)
name	VARCHAR(100)	Display name
avatar_url	VARCHAR	Profile picture
plan	ENUM('free','pro','admin')	Subscription tier
plan_expires_at	TIMESTAMPTZ	Subscription expiry
onboarding_done	BOOLEAN DEFAULT false	Onboarding completion flag
created_at	TIMESTAMPTZ DEFAULT now()	
updated_at	TIMESTAMPTZ	

Table: user_profiles

Column	Type	Notes
id	UUID PK	
user_id	UUID FK -> users.id	1:1 with users
dietary_goals	JSONB	calories, protein, carbs, fat targets
allergies	TEXT[]	e.g. ['gluten','nuts']
diets	TEXT[]	e.g. ['vegan','keto']
cooking_skill	ENUM('beginner','intermediate','advanced')	
household_size	SMALLINT DEFAULT 1	Number of people
preferred_cuisines	TEXT[]	e.g. ['italian','asian']
disliked_ingredients	TEXT[]	Excluded from AI suggestions

Table: recipes

Column	Type	Notes
id	UUID PK	
title	VARCHAR(255)	Recipe name
slug	VARCHAR(255) UNIQUE	URL-friendly identifier
description	TEXT	Short description
author_id	UUID FK -> users.id	Nullable = official recipe
source	ENUM('user','ai','official')	Origin of recipe
prep_time_min	SMALLINT	Preparation time in minutes
cook_time_min	SMALLINT	Cooking time in minutes
servings	SMALLINT	Default serving count
calories_per_serving	DECIMAL(8,2)	
protein_g	DECIMAL(8,2)	Grams per serving
carbs_g	DECIMAL(8,2)	Grams per serving
fat_g	DECIMAL(8,2)	Grams per serving
image_url	VARCHAR	S3 URL
is_public	BOOLEAN DEFAULT true	
is_approved	BOOLEAN DEFAULT false	Admin moderation flag
tags	TEXT[]	e.g. ['keto','quick','gluten-free']
cuisine	VARCHAR(50)	e.g. 'italian'
meal_type	TEXT[]	['breakfast','lunch','dinner','snack']
created_at	TIMESTAMPTZ DEFAULT now()	

Table: recipe_ingredients

Column	Type	Notes
id	UUID PK	
recipe_id	UUID FK -> recipes.id	CASCADE delete
name	VARCHAR(150)	Ingredient name
quantity	DECIMAL(10,3)	Amount
unit	VARCHAR(50)	e.g. 'g', 'cup', 'tbsp'
notes	VARCHAR(200)	e.g. 'finely chopped'
order_index	SMALLINT	Display order

Table: meal_plans

Column	Type	Notes
id	UUID PK	
user_id	UUID FK -> users.id	Plan owner
week_start	DATE	Monday of the plan week
generated_by	ENUM('manual', 'ai')	Creation method
ai_prompt_summary	TEXT	What AI was asked (for replay)
locked	BOOLEAN DEFAULT false	Prevents overwrite
created_at	TIMESTAMP TZ DEFAULT now()	

Table: meal_plan_entries

Column	Type	Notes
id	UUID PK	
plan_id	UUID FK -> meal_plans.id	CASCADE delete
recipe_id	UUID FK -> recipes.id	
day_of_week	SMALLINT (0=Mon...6=Sun)	
meal_type	ENUM('breakfast', 'lunch', 'dinner', 'snack')	
servings	DECIMAL(4,2) DEFAULT 1	Override default servings

Table: shopping_lists

Column	Type	Notes
id	UUID PK	
plan_id	UUID FK -> meal_plans.id	Linked meal plan
user_id	UUID FK -> users.id	
generated_at	TIMESTAMPTZ	Last generation timestamp

Table: shopping_list_items

Column	Type	Notes
id	UUID PK	
list_id	UUID FK -> shopping_lists.id	CASCADE delete
ingredient_name	VARCHAR(150)	Consolidated ingredient
total_quantity	DECIMAL(10,3)	Summed across recipes
unit	VARCHAR(50)	
category	VARCHAR(80)	e.g. 'Dairy', 'Vegetables'
is_checked	BOOLEAN DEFAULT false	User ticked in app

Table: subscriptions

Column	Type	Notes
id	UUID PK	
user_id	UUID FK -> users.id	
provider	ENUM('stripe','apple','google')	Payment provider
provider_sub_id	VARCHAR	External subscription ID
status	ENUM('active','cancelled','past_due','trialing')	
current_period_end	TIMESTAMPTZ	
created_at	TIMESTAMPTZ DEFAULT now()	

Table: ai_usage_logs

Column	Type	Notes
id	UUID PK	
user_id	UUID FK -> users.id	
type	ENUM('meal_plan','recipe','nutrition')	Request type
tokens_used	INTEGER	Total tokens consumed
cost_usd	DECIMAL(10,6)	Approximate cost
latency_ms	INTEGER	Response latency
created_at	TIMESTAMPTZ DEFAULT now()	

5. API Endpoints

Base URL: <https://api.menumate.app/v1>

Auth

Method	Path	Description	Access
POST	/auth/register	Register with email+password	Public
POST	/auth/login	Login, returns JWT + refresh token	Public
POST	/auth/refresh	Refresh access token	Public
POST	/auth/logout	Invalidate refresh token	Auth
GET	/auth/me	Get current user profile	Auth
POST	/auth/oauth/:provider	OAuth callback (Google, Apple)	Public
POST	/auth/forgot-password	Send reset email	Public
POST	/auth/reset-password	Reset with token	Public

Users & Profiles

Method	Path	Description	Access
GET	/users/profile	Get full profile + dietary prefs	Auth
PUT	/users/profile	Update profile	Auth
PATCH	/users/dietary	Update dietary goals/allergies	Auth
DELETE	/users/account	Delete account + all data	Auth
GET	/users/stats	Nutrition stats, plan history	Auth

Recipes

Method	Path	Description	Access
GET	/recipes	List recipes (filter, paginate, search)	Public
GET	/recipes/:id	Get single recipe detail	Public
POST	/recipes	Create recipe	Auth
PUT	/recipes/:id	Update recipe (owner only)	Auth
DELETE	/recipes/:id	Delete recipe (owner only)	Auth
GET	/recipes/search	Full-text search with filters	Public
POST	/recipes/:id/bookmark	Save recipe to favourites	Auth
DELETE	/recipes/:id/bookmark	Remove from favourites	Auth
GET	/recipes/bookmarks	List bookmarked recipes	Auth
POST	/recipes/scan	Barcode scan -> ingredient lookup	Auth

Meal Plans

Method	Path	Description	Access
GET	/meal-plans	List user's meal plans	Auth
GET	/meal-plans/:id	Get single plan with entries	Auth
POST	/meal-plans	Create manual meal plan	Auth
PUT	/meal-plans/:id	Update plan (name, lock)	Auth
DELETE	/meal-plans/:id	Delete plan	Auth
POST	/meal-plans/:id/entries	Add recipe to plan slot	Auth
DELETE	/meal-plans/:id/entries/:entryId	Remove entry from slot	Auth
POST	/meal-plans/ai/generate	AI-generate full week plan	Pro
POST	/meal-plans/ai/suggest	AI suggest recipe for one slot	Auth
GET	/meal-plans/:id/nutrition	Aggregated nutrition summary	Auth

Shopping Lists

Method	Path	Description	Access
GET	/shopping-lists/:planId	Get (or auto-generate) list for plan	Auth
POST	/shopping-lists/generate	Regenerate from current plan	Auth
PATCH	/shopping-lists/:id/items/:itemId	Toggle item checked	Auth
POST	/shopping-lists/:id/items	Add custom item	Auth
DELETE	/shopping-lists/:id/items/:itemId	Remove item	Auth

Subscriptions

Method	Path	Description	Access
GET	/subscriptions/plans	List available pricing plans	Public
POST	/subscriptions/checkout	Create Stripe checkout session	Auth
POST	/subscriptions/portal	Stripe billing portal URL	Auth
POST	/subscriptions/webhook	Stripe / Apple / Google webhook	System
GET	/subscriptions/status	Current subscription status	Auth

Admin

GET	/admin/users	List all users (search, filter)	Admin
GET	/admin/users/:id	User detail + activity	Admin
PATCH	/admin/users/:id/ban	Ban / unban user	Admin
GET	/admin/recipes/pending	Recipes awaiting moderation	Admin
PATCH	/admin/recipes/:id/approve	Approve recipe	Admin
DELETE	/admin/recipes/:id	Remove recipe	Admin
GET	/admin/analytics	DAU, MAU, conversion stats	Admin
GET	/admin/ai-logs	AI usage + cost dashboard	Admin
POST	/admin/notifications/broadcast	Send push/email to segment	Admin

6. Admin Panel — Scope

The admin panel is a separate Next.js app (apps/admin) protected by role-based access control. Only users with plan='admin' can access it.

Module	Description
Dashboard	KPI overview: DAU/MAU, new signups today, revenue MRR, AI cost this month, recipes pending review
User Management	Search, filter, view full activity. Ban/unban. Manually upgrade/downgrade subscription tier
Recipe Moderation	Queue of user-submitted recipes. Preview, approve, reject with reason. Bulk actions
Content Management	Edit diet tags, cuisines, featured collections. Seasonal homepage banners
Subscriptions & Revenue	View all transactions. Issue refunds via Stripe dashboard link. Manage promo codes
Push / Email Campaigns	Compose broadcast to segment (all free users, churned pro users, etc.). Schedule sends
AI Usage Monitor	Per-user token consumption, daily cost chart, error rate, slow queries log
Analytics	Retention cohorts, recipe popularity, conversion funnel free→pro, feature usage heatmap

Tech for Admin Panel

- Next.js 15 App Router — same stack as web app, shared UI package
- NextAuth.js middleware guard — redirect if role != admin
- Recharts — DAU/MAU, revenue, AI cost charts
- TanStack Table — sortable/filterable user and recipe tables
- Deployed separately (admin.menumate.app) — different Vercel project

7. AI Integration

MenuMate uses **Anthropic Claude API** (claude-sonnet-4-20250514) for all AI features. Requests are queued via Bull+Redis to avoid blocking HTTP responses.

Weekly meal plan generation **[Pro]**

Takes user profile (dietary goals, allergies, household size, preferred cuisines, disliked ingredients, cooking skill) and generates a full 7-day plan with breakfast/lunch/dinner/snack slots. Returns structured JSON parsed and saved to DB.

Single-slot recipe suggestion **[Free]**

Suggests 3 recipes for a specific meal slot (day + meal type) based on what's already in the plan (to avoid repetition) and user preferences.

Recipe from fridge (pantry AI) **[Pro]**

User enters available ingredients. AI generates a custom recipe that uses them, respecting dietary restrictions. Recipe is saved to user's private collection.

Nutritional analysis **[Free]**

Given a custom meal description, estimates calories, protein, carbs, fat. Useful for meals not in the recipe database.

Shopping list optimisation **[Free]**

Consolidates ingredients from weekly plan, groups by supermarket aisle, suggests quantity rounding to standard package sizes.

8. Environment & Deployment

Variable	Example	Purpose
DATABASE_URL	postgresql://...	Prisma connection string
JWT_SECRET	random 64-char hex	Access token signing
JWT_REFRESH_SECRET	random 64-char hex	Refresh token signing
ANTHROPIC_API_KEY	sk-ant-...	Claude API access
STRIPE_SECRET_KEY	sk_live_...	Stripe payments
STRIPE_WEBHOOK_SECRET	whsec_...	Webhook verification
REDIS_URL	redis://...	Bull queue + rate limiting
AWS_S3_BUCKET	menumate-uploads	Image storage
AWS_ACCESS_KEY_ID	...	S3 credentials
NEXTAUTH_SECRET	random 32-char	NextAuth session signing
EXPO_PUBLIC_API_URL	https://api.menumate.app/v1	Mobile API base
FCM_SERVER_KEY	...	Firebase push (Android)
APPLE_TEAM_ID	...	APNs push (iOS)

Deployment targets

App	Platform	Notes
web (Next.js)	Vercel	Auto-deploy on push to main
admin (Next.js)	Vercel	Separate project, protected domain
mobile (Expo)	EAS Build + Expo Updates	OTA updates for JS bundle
api (Express)	Railway or Render	Dockerised, auto-scale
PostgreSQL	Supabase or Railway	Managed, daily backups
Redis	Upstash	Serverless Redis, pay-per-use
S3 images	AWS S3 + CloudFront	CDN for recipe images

Getting started locally

- `pnpm install` (installs all workspaces)
- `cp .env.example .env` (fill in secrets)
- `pnpm db:migrate` (runs Prisma migrations)
- `pnpm db:seed` (loads sample recipes)
- `pnpm dev` (starts all apps concurrently via Turborepo)
- Web: <http://localhost:3000> | Admin: <http://localhost:3001> | API: <http://localhost:4000>